

Enter through evidence. Earn leverage through closure.

The plan is designed to learn Latitude's actual architecture, toolchain, integration rhythm, and decision boundaries before prescribing changes. Each phase increases direct contribution while preserving domain ownership.

DAYS 0-30 | LEARN THE SYSTEM

Technical immersion
Set up the C++/Python build and test environment; learn repository structure, platform abstractions, configuration management, telemetry, validation, on-vehicle compute constraints, and safety interfaces.

Integration flow map
Trace one feature or platform change from configuration through compute/network, sensors, autonomy software, test evidence, release posture, and ownership handoff.

Failure evidence map
Review representative bring-up, regression, intermittent, and configuration issues. Identify how symptoms become hypotheses, how evidence is captured, and where ambiguity persists.

Relationship map
Pair with autonomy, systems, safety, test, platform, compute, networking, sensor, and configuration owners. Learn who decides, who contributes, and what blocks release.

First bounded contribution
Own a contained integration issue or tooling improvement with a senior partner; document assumptions, evidence, experiments, repair, and durable closure mechanism.

30-DAY OUTPUTS

- 01**
System and stakeholder map
- 02**
Current-state closure workflow
- 03**
Configuration/evidence inventory
- 04**
One bounded issue or tool improvement
- 05**
Questions and hypotheses for validation

DAYS 31-60 | IMPROVE THE SIGNAL

Own bounded integration work

Drive selected issues from reproduction through closure, keeping the next experiment, evidence gap, owner, and release posture explicit.

Strengthen observability

Improve synchronized logs, trace correlation, platform/configuration identifiers, or diagnostics where missing evidence forces guesswork.

Make configuration comparable

Support machine-readable comparison of software, firmware, calibration, network, environment, and platform identities against known-good references.

Improve blocker communication

Use concise integration readouts that state symptom, invariant, evidence, hypotheses, next experiment, owner, and release impact.

Convert closure into protection

Add targeted regression checks, compatibility controls, validation evidence, or interface documentation for repeated failure patterns.

DAYS 61-90 | STRENGTHEN THE MECHANISM

Pattern review

Cluster recurring integration failures by violated invariant, missing evidence, configuration drift, ownership boundary, and escaped-regression mechanism.

Reusable closure mechanisms

Propose a small set of high-leverage checks, templates, dashboards, or tooling changes grounded in observed work.

Integration health view

Make blocker age, reproducibility, configuration traceability, release evidence, escaped regression, and recurrence visible without inventing vanity metrics.

Platform readiness support

Apply the model to one new-platform or major-feature integration path and document what should become easier for the next rollout.

MEASURES AND OPERATING PRINCIPLES

Measure categories

Time to bounded hypothesis
Minimum reproducible condition quality
Blocker age and next-experiment clarity
Good configuration lineage and provenance
Escaped regressions and recurrence
Reuse of closure mechanisms

Operating principles

Evidence over ego
Pair before prescribing
Respect domain ownership
Automate repeatable checks
Make release posture explicit
Leave the system easier to explain